# Analysis of Massive Data Sets

http://www.fer.hr/predmet/avsp

**Prof. dr. sc. Siniša Srbljić**
**Doc. dr. sc. Marin Šilić**
**Doc. dr. sc. Goran Delač**
**Doc. dr. sc. Klemo Vladimir**
**Adrian Satja Kurdija, mag. inf. et math.**

Faculty of Electrical Engineering and Computing
Consumer Computing Laboratory

Copyright © 2017 CCL : Analysis of massive data sets

# Mining Data Streams (lecture #2)

Faculty of Electrical Engineering and Computing
Consumer Computing Laboratory

# Mining Data Streams

- Outline
  1. Introduction/Stream Data Model
  2. Sampling streams (*Random sample/Sliding windows*)
  3. Filtering streams (*Bloom filter*)
  4. Counting distinct elements (*Flajolet-Martin*)
  5. **Estimating moments (*Alon-Matias-Szegedy*)**
  6. **Counting 1's (*Datar-Gionis-Indyk-Motwani*)**
  7. **Decaying windows**
  8. Literature

Copyright © 2017 CCL : Analysis of massive data sets

# 5. Estimating moments

Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

- Moments
  - Generalization of the *count distinct* problem
  - Stream has elements chosen from a set A of n values
  - $m_i$ = number of occurrences of the *i*th element for any i
  - ***k*th-order moment** of the stream
    - Sum over all i of $(m_i)^k$

$$\sum_{i \in A} m_i^k$$

Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

- Special cases
  - $0^{th}$ moment
    - Number of distinct elements
  - $1^{st}$ moment
    - Length of the stream
  - $2^{nd}$ moment
    - *Surprise number*
    - Measure of how uneven the distribution is

Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

- Special cases
  - 2nd moment
    - *Surprise number*
    - Example
      - Stream length: 100
      - 11 values appear
        - 10, 9, 9, 9, 9, 9, 9, 9, 9, 9, 9 = 910
        - 90, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1 = 8110

Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

- Alon-Matias-Szegedy
  - Works for all moments
  - Unbiased
  - Based on calculation of many random variables

# Estimating moments

- Alon-Matias-Szegedy
  - $2^{nd}$ moment
    - $\sum_a m_a^2$
    - Suppose there is not enough space to count all the m's
    - Estimation using limited space
      - Compute some number of variables
      - For each variable **X** store
        - **X.element**
        - **X.value**

# Estimating moments

- Alon-Matias-Szegedy
  - For each **X**
    - 1. choose position in the stream randomly
    - 2. set **X.element** to be the element found at that position
    - 3. initialize **X.value** to 1
    - 4. add **1** to **X.value** each time another occurrence of **X.element** is encountered

Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

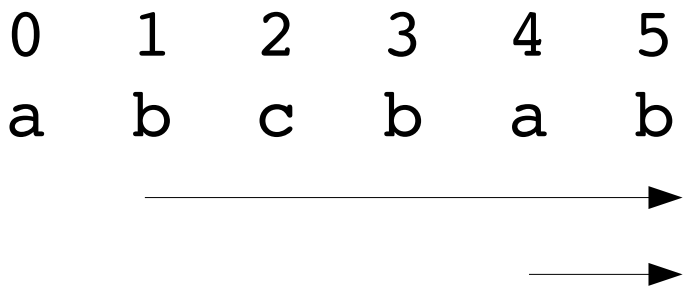
- Alon-Matias-Szegedy
  - $2^{nd}$ moment estimate

    **n(2X.value − 1)**
  - For any X
  - Example
    - Stream: a, b, c, b, a, b
    - n = 6
    - $m_a = 2$, $m_b = 3$, $m_c = 1$
    - $2^{nd}$ moment = $1^2 + 2^2 + 3^2 = 14$

Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

- Alon-Matias-Szegedy
  - Stream: a, b, c, b, a, b
  - 2 variables ($X_1$ and $X_2$)
  - Random positions: 1, 4
    - From position 1:
      - $X_1$.element = b, $x_1$.value = 3
    - From position 4:
      - $X_2$.element = a, $x_2$.value = 1
  - 2nd moment estimate
    - $n(2X_1.value - 1) = 6(2*3 - 1) = 30$
    - $n(2X_2.value - 1) = 6(2*1 - 1) = 6$
    - Avg: 18

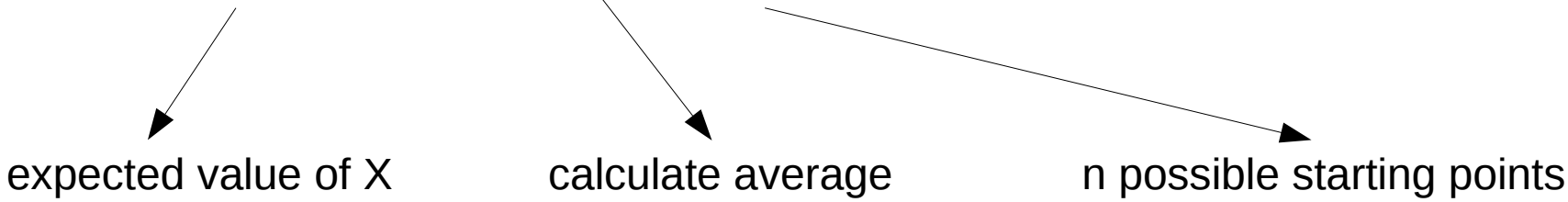



Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

- Alon-Matias-Szegedy
  - Why it works?
    - e(i) = stream element that appears at position i
    - c(i) = number of times element e(i) appears in the rest of the stream

$$E(n(2x-1)) = 1/n \sum_{1..n} n(2c(i)-1)$$




Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

- Alon-Matias-Szegedy
  - Why it works?
    - e(i) = stream element that appears at position i
    - c(i) = number of times element e(i) apperts in the rest of the stream

$$E(n(2x-1)) = 1/n \sum_{1..n} n(2c(i)-1)$$

$$= \sum_{1..n} (2c(i)-1)$$

$$= \sum_{a} 1+3+5+...+(2m_a - 1)$$

Last *a*: 2*1 – 1 = 1

Second last *a:* 2*2 – 1 = 3

First *a*: $2*m_a - 1$

Copyright © 2017 CCL : Analysis of massive data sets

# Estimating moments

- Alon-Matias-Szegedy
  - Why it works?
    - e(i) = stream element that appears at position i
    - c(i) = number of times element e(i) apperts in the rest of the stream

$$E(n(2x-1)) = 1/n \sum_{1..n} n(2c(i)-1)$$

$$= \sum_{1..n} (2c(i)-1)$$

$$= \sum_{a} 1+3+5+...+(2m_a - 1)$$

$$= \sum_{a} m_a^2$$

Copyright © 2017 CCL : Analysis of massive data sets

# Higher-Order Moments

- We estimate $k^{th}$ moments, for $k > 2$, in a similar way as we estimate second moments
- The only thing that changes is the way we derive an estimate from a variable
- $2v - 1$ is the difference between $v^2$ and $(v - 1)^2$
- For $3^{rd}$ moment, use $v^3 - (v-1)^3$ instead
- More generally, we can estimate $k^{th}$ moments for any $k \geq 2$ by turning value $v$ = X.value into

$$n(v^k - (v - 1)^k).$$

Copyright © 2017 CCL : Analysis of massive data sets

# Moments in Infinite Streams

- In practice, n grows with time
- Then we must be careful how we select the positions for the variables
- Early selection is biased in favour of early positions --> too large estimate of the moment
- Solution:
  - maintain as many variables as we can store
  - replace them with new ones as the stream grows

Copyright © 2017 CCL : Analysis of massive data sets

# Moments in Infinite Streams

- Suppose we have space to store **s** variables
- The first **s** positions of the stream are initially picked as variables
- If the current stream lenght is n, probability of any particular position being the position of a variable is uniform (s / n)
- When the $(n+1)^{st}$ element arrives, pick that position with probability s / (n+1)
- If the new position is picked as a variable, randomly discard one of the current **s** variables
- Probability remains uniform (prove it!)

Copyright © 2017 CCL : Analysis of massive data sets

# General Stream-Sampling

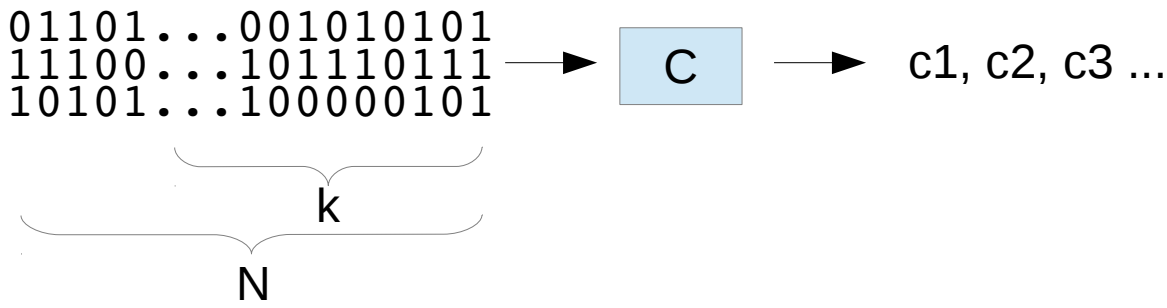
- This sampling techinque is a general way to maintain a sample of **s** stream elements so that at all times, all stream elements are equally likely to be selected
- Example: sampling users in search engine queries (from lecture #1) to study typical user behaviour

Copyright © 2017 CCL : Analysis of massive data sets

# 6. Counting 1’s

# Counting 1's

- Problem
  - Given a stream of zeroes (0's) and ones (1's),
  - Count a number of ones in the last *k* bits
    - k <= N
- Obvious solution
  - Store all N bits
    - When new bit arrives, remove oldest bit
  - Performance
    - Query takes O(k) time
    - Space-inefficient since N can be large (or many streams)




Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- Challenge
  - Count number of 1's with much less space/time requirements
  - Cannot afford to store all of the bits
- Exact solution
  - Exact count is not possible without all of the bits
- Approximation method
  - 1. Exponentially increasing blocks
  - 2. Datar-Gionis-Indyk-Motwani (DGIM) algorithm

Copyright © 2017 CCL : Analysis of massive data sets

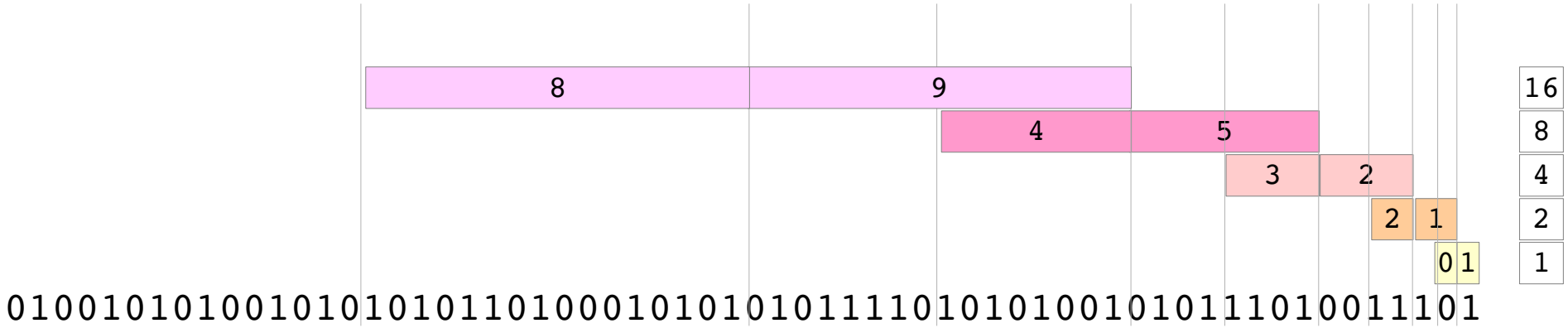
# Counting 1's

- Exponentially increasing blocks
  - Summarize exponentially increasing blocks of the stream, looking backward
    - 1, 2, 4, 8, 16, …
  - Summary is the number of ones in the block
  - Keep never more than two blocks of any size

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

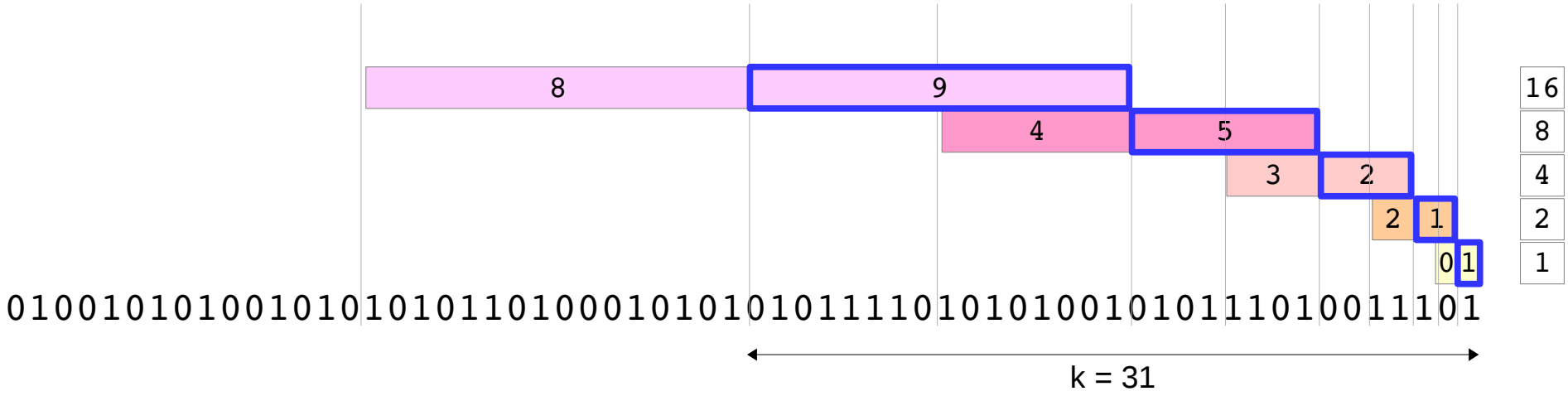
- Exponentially increasing blocks
  - two blocks of any size




Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- Exponentially increasing blocks
  - Query: count 1's in the last **k=31** bits

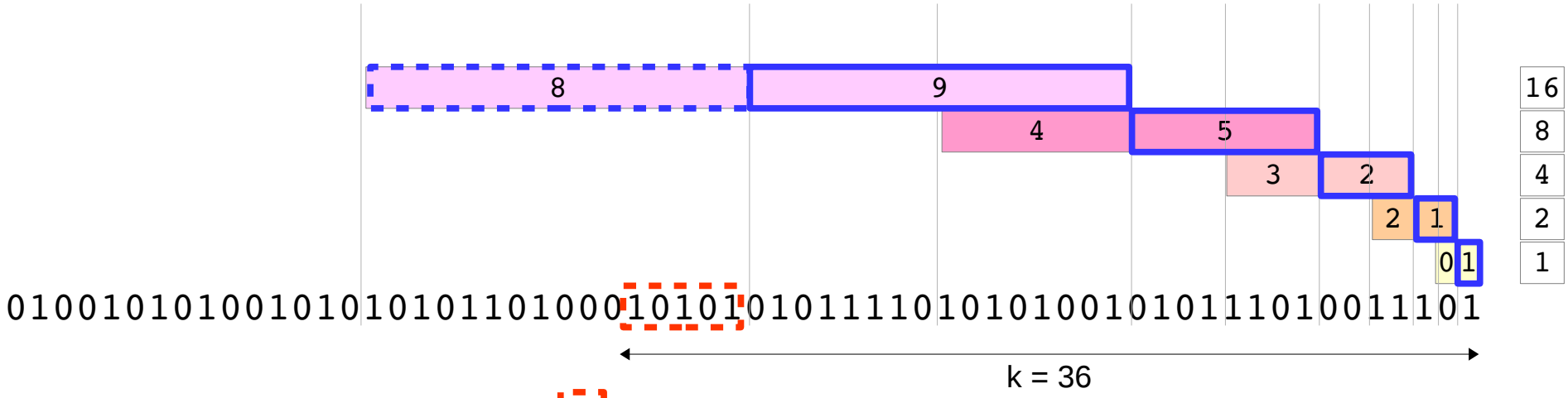



Answer: 1 + 1 + 2 + 5 + 9 = 18

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- Exponentially increasing blocks
  - Query: count 1's in the last **k=36** bits




Answer: 1 + 1 + 2 + 5 + 9 + 3 = 21

Estimate: 1 + 1 + 2 + 5 + 9 + x = ? x = 8/2=4 (half estimate), or
x = 8/16*5=2.5 (prop. guess)

Copyright © 2017 CCL : Analysis of massive data sets

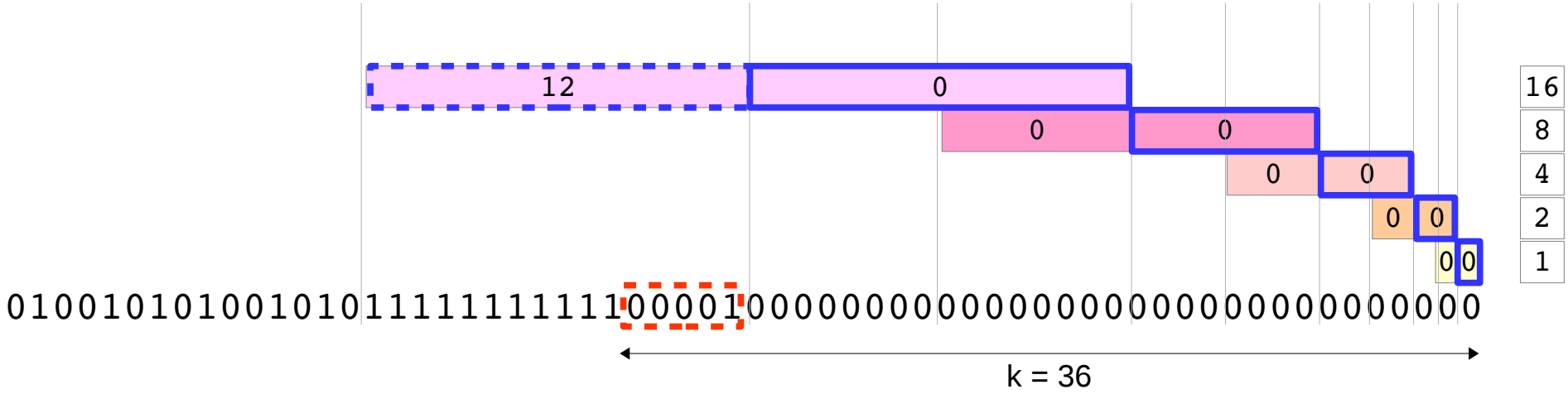
# Counting 1's

- Exponentially increasing blocks
  - Store 2 * $\log_2 N$ * $\log_2 N$ bits = $O(\log^2 N)$
    - First $\log_2 N$ for blocks (2 of each size)
    - Second $\log_2 N$ for the counter
  - Error
    - No greater than the number of zeroes in the last bits that are not covered by complete block
    - Depends on the distribution of 1's

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- Exponentially increasing blocks
  - Error
    - All the 1's are in the last bits that are not covered by complete block




Estimate: 0 + x = ?  x = 12/2 = 6 (Right answer: **1** ones) Error: **600%**

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- Datar-Gionis-Indyk-Motwani (DGIM) algorithm
  - Similar to the previous algorithm
  - Avoids the problem of uneven distribution of 1's
  - Instead of fixed-length blocks, keep blocks with specific number of 1's
    - Exponential block sizes (size = num of 1's)
  - Stores $O(\log^2 N)$ bits per stream
  - Approximate answer
    - Error max 50% true count

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- DGIM algorithm
  - Timestamps
    - Each bit has timestamp (position): 1, 2, 3, ...
    - Timestamp is modulo N (window size)
      - need **$\log_2 N$ bits**
  - Buckets
    - Segment of the window defined by
      - Timestamp of it's end ($\log_2 N$ bits)
      - (Power of 2) number of 1's in the bucket
        - X = 1, 2, 4, 8, … → $2^0$, $2^1$, $2^2$, $2^3$, … → $2^y$
        - max(y) = $\log_2 N$
        - Bucket memory representation → **$\log_2 \log_2 N$ bits**
      - Total: **O(logN) bits** for the bucket

Copyright © 2017 CCL : Analysis of massive data sets

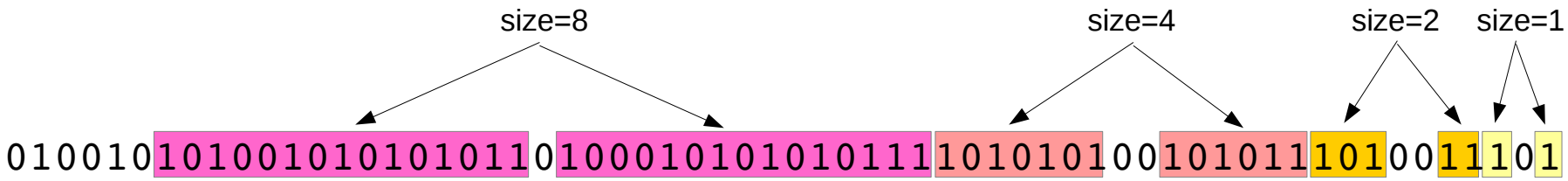
# Counting 1's

- DGIM algorithm
  - Total storage requirements
    - Window length → N
    - Largest bucket size → $2^y$
      - $y < \log_2 N$
    - Bucket sizes: 1 … $\log_2 N$
  - Number of bits needed
    - O(logN) buckets * O(logN) bits per bucket
      - **$O(\log^2 N)$**

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- DGIM algorithm
  - Basic idea
    - Size of the bucket is power of 2 number of 1's in the bucket




Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- DGIM algorithm
  - Rules
    - Every position with a 1 is in some bucket (no more than one)
    - One or two buckets with the same size
    - Sizes are power of 2
    - Buckets are sorted by size and do not overlap

Copyright © 2017 CCL : Analysis of massive data sets

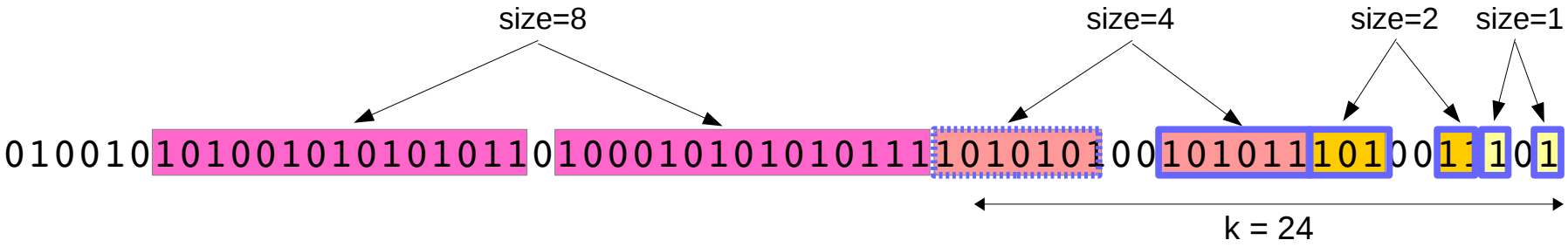
# Counting 1's

- DGIM algorithm
  - Query answering
    - How many 1's there are in the last k bits ($k \leq N$)
  - Procedure (O(logN))
    - Sum sizes of all buckets but the last
      - Last bucket → bucket with the earliest timestamp that includes at least some of the k most recent bits
    - Add half the size of the last bucket

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- DGIM algorithm
  - Query answering
    - k=24




Answer: 1 + 1 + 2 + 2 + 4 + 4/2 = 12 (True count: 14)

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- DGIM algorithm
  - Maintaining buckets
    - When a new bit comes in
      - delete the oldest bucket if its end-time is prior to N time units before the current time
      - If the new bit is 0 → no other changes
      - If the new bit is is 1
        - Create new bucket with size 1 (for the new bit) and current timestamp
        - If there are 3 buckets of size 1 → merge oldest two into single bucket of size 2
        - If there are 3 buckets of size 2 → merge oldest two into single bucket of size 4
        - …

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

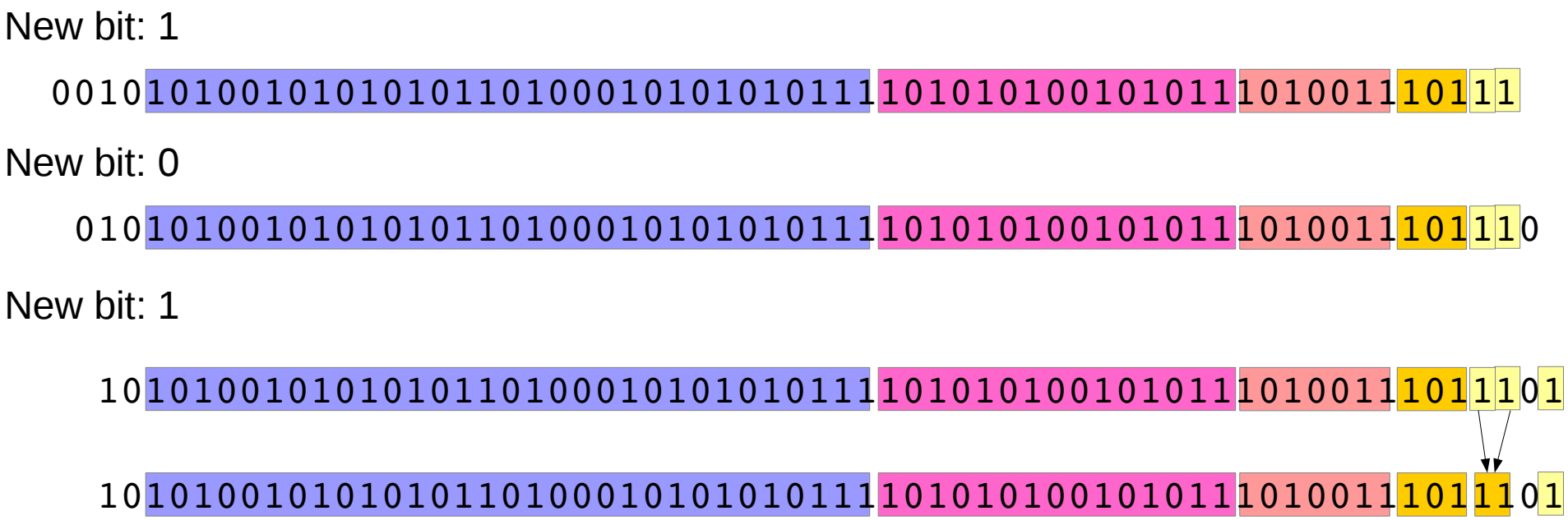
- DGIM algorithm
  - Maintaining buckets




Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

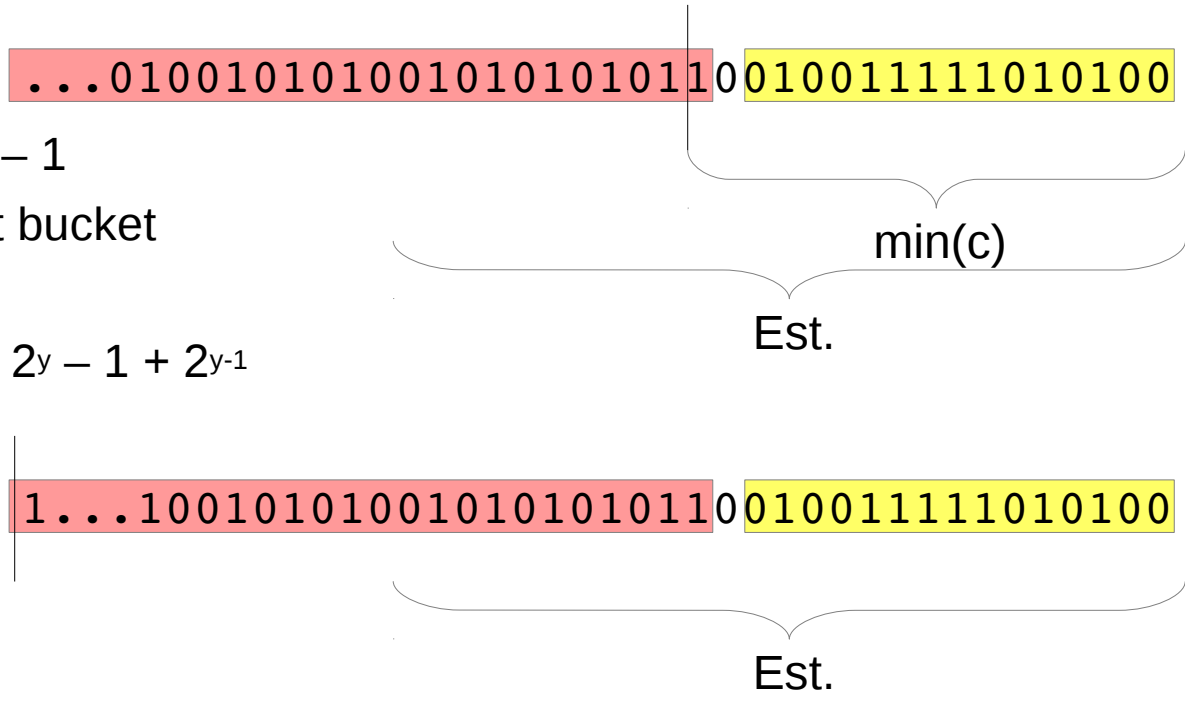
- DGIM algorithm
  - Maintaining buckets




Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- DGIM algorithm - Error analysis
  - Last bucket ($2^y$) approximation is x/2 → $2^{y-1}$
  - Existing bucket sizes → 1 … $2^{y-1}$
    - True count → c
  - a) **Estimate is greater than c**
    - min(c) = 1 + 2 + 4 + … + $2^{y-1}$ = $2^y$ – 1
    - Add 1 from the single 1 in the last bucket
    - min(c) = $2^y$ – 1 + 1 = $2^y$
    - Estimate: 1 + 2 + … + $2^{y-1}$ + $2^{y-1}$ = $2^y$ – 1 + $2^{y-1}$
    - Est is max 1.5 * min(c)
  - b) **Estimate is less than c**
    - All of the 1's are in the range
    - Estimate misses half ($2^{y-1}$) bits
    - min(c) = 1 + 2 + … + $2^{y-1}$ + $2^y$ = $2^{y+1}$ – 1
    - Min Est is 0.5 * min(c)




Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's

- Generalization
  - Allow **more than two** buckets of any size (up to *r*)
  - More buckets of smaller sizes → stronger bound on the error
  - $r = \text{ceil}(1/\varepsilon)$
  - Memory requirements: $O(r * \log^2 N)$
  - Algorithm update:
    - Merge if there are $r + 1$ buckets of the same size
  - Error
    - Estimate is within factor $1 + \varepsilon$
      - Simplest case: $r = 2$ → error 50%

Copyright © 2017 CCL : Analysis of massive data sets

# Counting 1's - Extension

- Consider streams of **integers**
- We want to estimate the **sum** of last k integers for any $1 \leq k \leq N$
- DGIM does not work in case of negative integers
- If integers belong to $[0, 2^m - 1]$, each can be represented by m bits
- Treat each bit-position as a separate stream
- Use DGIM for each stream and sum up the results with corresponding factors:

$$\text{estimate} = \text{sum}(\text{estimate}(i) * 2^i)$$

Copyright © 2017 CCL : Analysis of massive data sets

# 7. Decaying Windows

Copyright © 2017 CCL : Analysis of massive data sets

# Decaying Windows

- Sometimes we do not want to make a sharp distinction between the N most recent elements and those in the more distant past
- We just want to weight the recent elements more heavily (continuum instead of a cut)
- Finding the most popular elements:
  - Stream of sold tickets to movies
  - We want to estimate current movie popularities (recent tickets weight more)

Copyright © 2017 CCL : Analysis of massive data sets

# Decaying Windows

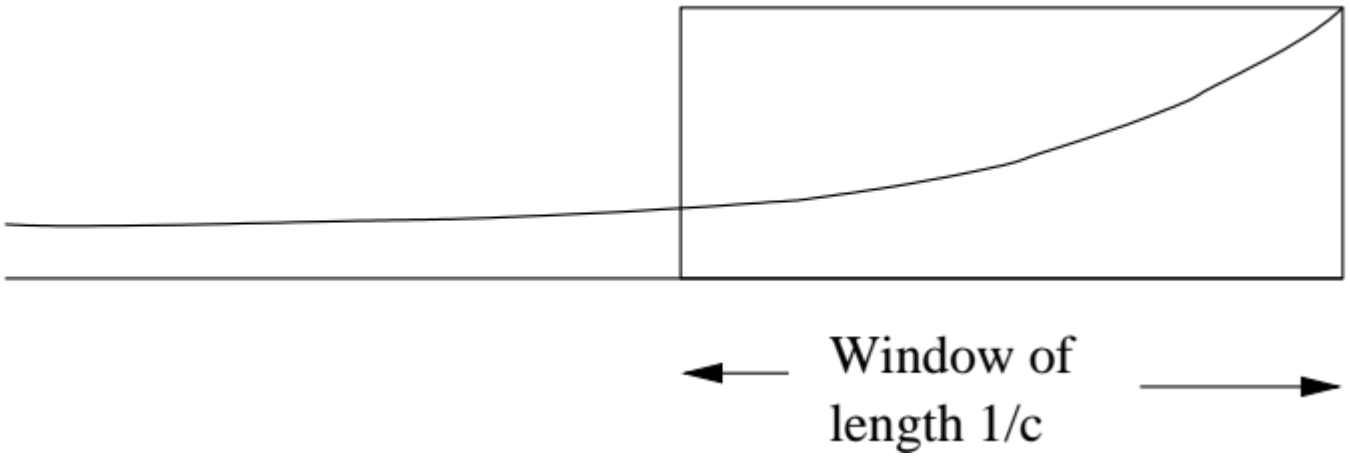
- Imagine a bit stream for each movie
- We could use DGIM (for some N)
  - Problems: sharp cut, how to choose N and k?
  - Efficiency problems when the number of streams is huge
- Exponentially decaying window:
  - Let *t* be the current stream lenght
  - Ticket at position *t - i* is worth $(1 - c)^i$ where *c* is a very small constant:

$$\sum_{i=0}^{t-1} a_{t-i}(1-c)^i$$

Copyright © 2017 CCL : Analysis of massive data sets

# Decaying vs. Fixed-Lenght Windows

- A fixed window with the same sum of weights (1/c) would put weight 1 on each of the most recent 1/c elements and 0 on all previous elements:




Copyright © 2017 CCL : Analysis of massive data sets

# Exponentially decaying window

- Much easier to adjust the sum than in a fixed-length sliding window
- When new element arrives:
  - Multiply the current sum by 1 - c,
  - Add the new element.
- Finding the most popular elements:
  - separate bit stream for each movie
  - drop movies with popularity less than e.g. 1/2
  - sum of all scores is 1/c --> at most 2/c movies are kept

Copyright © 2017 CCL : Analysis of massive data sets

# Literature


- J. Leskovec, A. Rajaraman, and J. D. Ullman, "**Mining of Massive Datasets**“, 2014, **Chapter 4. Mining Data Streams**
- P. Flajolet and G.N. Martin, “**Probabilistic counting for database applications**” 24th Symposium on Foundations of Computer Science, pp. 76–82, 1983.
- N. Alon, Y. Matias, and M. Szegedy, “**The space complexity of approximating frequency moments**” 28th ACM Symposium on Theory of Computing, pp. 20–29, 1996.
- Datar, Mayur, et al. "**Maintaining stream statistics over sliding windows.**" SIAM Journal on Computing 31.6 (2002): 1794-1813.


Copyright © 2017 CCL : Analysis of massive data sets